人体姿势识别

人体姿势识别
例程实验效果
代码讲解
核心代码

例程实验效果

本节我们来学习一下使用K230实现人体关键点检测的功能。

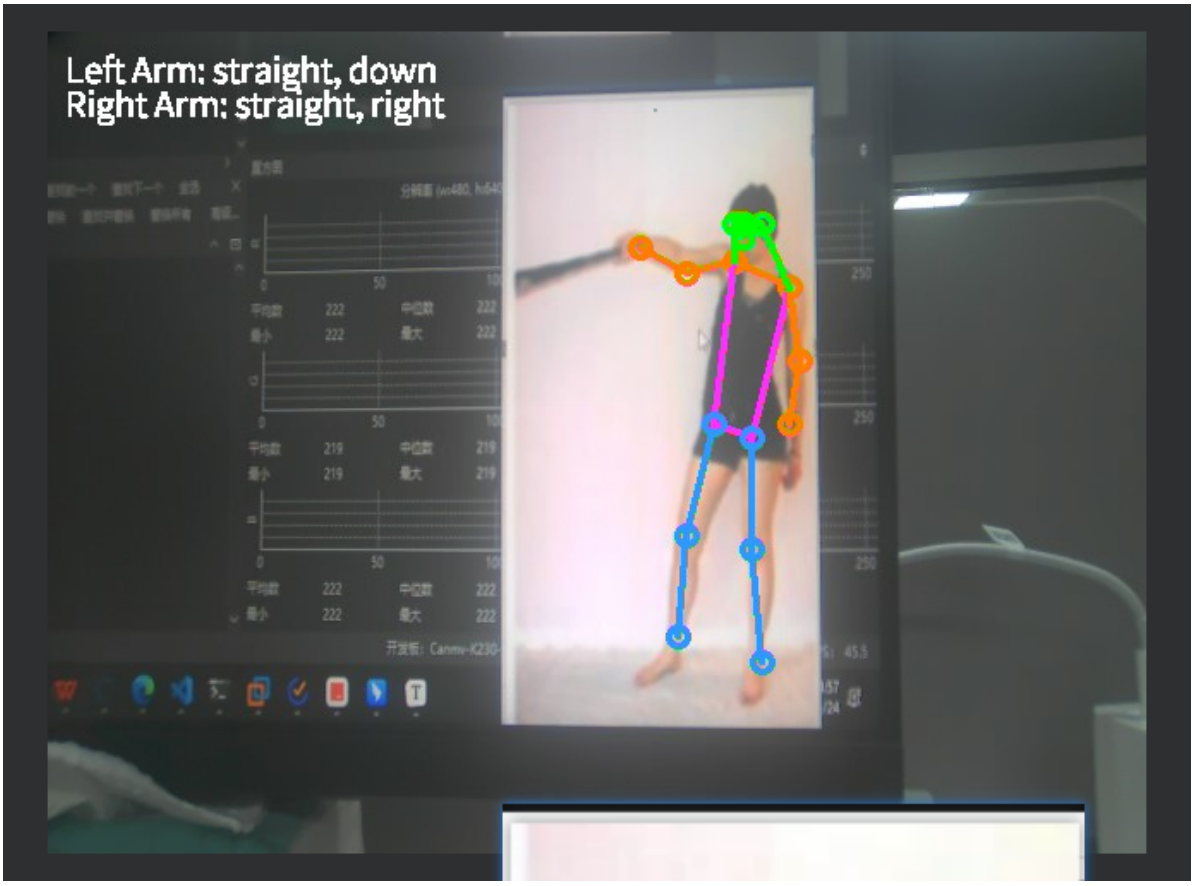
例程代码在【 源码汇总 / 08.Body / 03.person_gesture_cls.py】

连接IDE后运行本节例程代码，用K230对准一张有人体存在的画面，可以看到屏幕中标记出了人体关键点的位置。

通过对这些（胳膊）关键点的相对位置的比较，判断胳膊的状态:

> 状态包括伸直（straight），弯曲（bent），向左（left），向右（right）

判断的结果会显示在屏幕的左上角



代码讲解

我们主要分析判断姿势的思路

这段代码实现了人体姿势分析功能，主要分析手臂的状态（伸直/弯曲）和朝向。让我分模块详细讲解：

1. analyze_pose 函数

```
def analyze_pose(self, kpses):
```

这是主要分析函数，处理多人的姿势分析：

- 输入参数kpses包含多个人的关键点数据，格式为[人数, 关键点数, 3]
- 每个关键点包含(x,y,置信度)三个值
- 对每个检测到的人分别分析左右手臂
- 调用_analyze_arm分别处理左臂(5,7,9号关键点)和右臂(6,8,10号关键点)

2. _analyze_arm 函数

```
def _analyze_arm(self, shoulder, elbow, wrist):
```

这是核心分析函数，通过以下步骤分析单个手臂：

A. 有效性检查：

- 检查三个关键点(肩、肘、腕)的置信度是否>0
- 无效则返回"unknown"状态

B. 角度计算：

- 计算肩到肘和肘到腕的向量
- 使用向量点积计算两个向量间的夹角
- 通过arccos得到角度值

C. 状态判断：

- 角度<=60度判定为"straight"(伸直)
- 角度>60度判定为"bent"(弯曲)

D. 朝向判断：

- 比较肩到肘向量在x和y方向的分量
- 水平分量大则判断左/右
- 垂直分量大则判断上/下

3. draw_result 函数

```
def draw_result(self, pl, res, pose_results):
```

这是可视化函数：

- 绘制骨骼关键点连线
- 在每个检测到的人物上方显示姿势分析结果
- 包含坐标转换确保正确显示

关键技术：

1. 向量计算：使用numpy进行向量运算
2. 角度计算：使用点积和arccos计算夹角
3. 姿态判定：基于角度阈值和向量投影判断
4. 坐标变换：考虑显示尺寸进行坐标映射

核心代码

```
    def analyze_pose(self, kpses):
        """
        分析人体姿势，判断手臂的伸直/弯曲状态及朝向
        Analyze human pose, determine arm straight/bent state and direction

        Args:
            kpses: 关键点数据，格式为 [num_persons, num_keypoints, 3]，每个关键点包含 (x, y, confidence)
                   (Keypoint data in format [num_persons, num_keypoints, 3], each keypoint contains (x, y, confidence))
        Returns:
            pose_results: 包含每个人的姿势分析结果的列表 (List containing pose analysis results for each person)
        """
        pose_results = []
        for i in range(len(kpses)):  # 遍历每个检测到的人 (Iterate through each detected person)
            person_kps = kpses[i]
            person_pose = {}

            # 左臂分析（关键点 5: 左肩, 7: 左肘, 9: 左手腕）
            # Left arm analysis (keypoint 5: left shoulder, 7: left elbow, 9: left wrist)
            left_arm_result = self._analyze_arm(
                shoulder=person_kps[5],  # 关键点索引从1开始，数组从0开始 (Keypoint index starts from 1, array starts from 0)
                elbow=person_kps[7],
                wrist=person_kps[9]
            )
            person_pose["left_arm"] = left_arm_result

            # 右臂分析（关键点 6: 右肩, 8: 右肘, 10: 右手腕）
            # Right arm analysis (keypoint 6: right shoulder, 8: right elbow, 10: right wrist)
            right_arm_result = self._analyze_arm(
                shoulder=person_kps[6],
                elbow=person_kps[8],
                wrist=person_kps[10]
            )
            person_pose["right_arm"] = right_arm_result

            pose_results.append(person_pose)

        return pose_results

    def _analyze_arm(self, shoulder, elbow, wrist):
        """
        分析单条手臂的姿势
        Analyze the pose of a single arm

        Args:
            shoulder: 肩关节坐标 (x, y, confidence) (Shoulder joint coordinates (x, y, confidence))
            elbow: 肘关节坐标 (x, y, confidence) (Elbow joint coordinates (x, y, confidence))
```

```
            wrist: 手腕坐标 (x, y, confidence) (Wrist coordinates (x, y,
confidence))
        Returns:
            result: 包含伸直/弯曲状态和朝向的字典 (Dictionary containing
straight/bent state and direction)
        """
        result = {"state": "unknown", "direction": "unknown"}

        # 检查关键点是否有效（置信度 > 0）(Check if keypoints are valid (confidence >
0))
        if shoulder[2] <= 0 or elbow[2] <= 0 or wrist[2] <= 0:
            return result

        # 计算肩-肘向量和肘-腕向量 (Calculate shoulder-elbow vector and elbow-wrist
vector)
        vec_shoulder_to_elbow = np.array([elbow[0] - shoulder[0], elbow[1] -
shoulder[1]])
        vec_elbow_to_wrist = np.array([wrist[0] - elbow[0], wrist[1] -
elbow[1]])

        # 计算向量模长 (Calculate vector magnitudes)
        norm_se = np.linalg.norm(vec_shoulder_to_elbow)
        norm_ew = np.linalg.norm(vec_elbow_to_wrist)

        # 避免除零 (Avoid division by zero)
        if norm_se == 0 or norm_ew == 0:
            return result

        # 计算夹角（角度）(Calculate angle (in degrees))
        cos_theta = np.dot(vec_shoulder_to_elbow, vec_elbow_to_wrist) / (norm_se
* norm_ew)
        cos_theta = min(1.0, max(-1.0, cos_theta))  # 限制cos值范围 (Restrict cos
value range)
        angle = math.degrees(math.acos(cos_theta))

        # 判断手臂伸直或弯曲 (Determine if arm is straight or bent)
        if angle <= 60:
            result["state"] = "straight"
        elif angle > 60:
            result["state"] = "bent"

        # 判断手臂朝向 (Determine arm direction)
        dx = elbow[0] - shoulder[0]
        dy = elbow[1] - shoulder[1]
        if abs(dx) > abs(dy):
            # 水平方向为主 (Primarily horizontal direction)
            result["direction"] = "left" if dx > 0 else "right"
        else:
            # 垂直方向为主 (Primarily vertical direction)
            result["direction"] = "down" if dy > 0 else "up"

        return result

    def draw_result(self, pl, res, pose_results=None):
        """
        绘制结果，包括骨骼、关键点和姿势信息
        Draw results including skeleton, keypoints and pose information
```

```
        Args:
            pl: PipeLine实例 (PipeLine instance)
            res: 检测结果 (Detection results)
            pose_results: 姿势分析结果 (Pose analysis results)
        """
        with ScopedTiming("display_draw", self.debug_mode > 0):
            if res[0]:  # 如果检测到人 (If persons are detected)
                pl.osd_img.clear()
                kpses = res[1]
                for i in range(len(res[0])):
                    # 绘制关键点和骨骼（原有逻辑）(Draw keypoints and skeleton
(original logic))
                    for k in range(17+2):
                        if k < 17:
                            kps_x, kps_y, kps_s = round(kpses[i][k][0]),
round(kpses[i][k][1]), kpses[i][k][2]
                            kps_x1 = int(float(kps_x) * self.display_size[0] //
self.rgb888p_size[0])
                            kps_y1 = int(float(kps_y) * self.display_size[1] //
self.rgb888p_size[1])
                            if kps_s > 0:
                                pl.osd_img.draw_circle(kps_x1, kps_y1, 5,
self.KPS_COLORS[k], 4)

                        ske = self.SKELETON[k]
                        pos1_x, pos1_y = round(kpses[i][ske[0]-1][0]),
round(kpses[i][ske[0]-1][1])
                        pos1_x_ = int(float(pos1_x) * self.display_size[0] //
self.rgb888p_size[0])
                        pos1_y_ = int(float(pos1_y) * self.display_size[1] //
self.rgb888p_size[1])

                        pos2_x, pos2_y = round(kpses[i][(ske[1]-1)][0]),
round(kpses[i][(ske[1]-1)][1])
                        pos2_x_ = int(float(pos2_x) * self.display_size[0] //
self.rgb888p_size[0])
                        pos2_y_ = int(float(pos2_y) * self.display_size[1] //
self.rgb888p_size[1])

                        pos1_s, pos2_s = kpses[i][(ske[0]-1)][2], kpses[i]
[(ske[1]-1)][2]
                        if pos1_s > 0.0 and pos2_s > 0.0:
                            pl.osd_img.draw_line(pos1_x_, pos1_y_, pos2_x_,
pos2_y_, self.LIMB_COLORS[k], 4)

                    # 绘制姿势信息 (Draw pose information)
                    if pose_results and i < len(pose_results):
                        pose = pose_results[i]
                        # 显示左臂信息 (Display left arm information)
                        left_arm_text = f"Left Arm: {pose['left_arm']['state']},
{pose['left_arm']['direction']}"
                        pl.osd_img.draw_string_advanced(
                            10, 10 + i * 60, 20, left_arm_text, (255, 255, 255,
255)
                        )
                        # 显示右臂信息 (Display right arm information)
                        right_arm_text = f"Right Arm: {pose['right_arm']
['state']}, {pose['right_arm']['direction']}"
```

```
                        pl.osd_img.draw_string_advanced(
                            10, 30 + i * 60, 20, right_arm_text, (255, 255, 255,
255)
                        )
                gc.collect()
            else:
                pl.osd_img.clear()
```